

Contents

1 draft: text capture	1
1.1 outline	1
1.2 draft	1
1.2.1 TODO what is web scraping?	1

1 draft: text capture

1.1 outline

- web scraping
 - what is web scraping?
 - * html data
 - * network, TCP/IP
 - * standards
 - scraping congress.gov
 - *
- conclusion
 - eve sedgwick on reading practices?
 - Bowers vs Hardwick, Bostock v. Clayton County.

1.2 draft

1.2.1 **TODO** what is web scraping?

1. HTML

Web scraping is the extraction of information from websites. It most often involves working with data in HTML, though there are other forms that data takes which can be intercepted, such as from requests that are necessary to populate web pages.¹

2. Protocol

Web scraping is possible by manipulating HTTP protocols, which is the language or standard of the network.

¹Scraping XHR

Web scraping is, at its core, about mimicking a typical browser request. Using HTTP protocols, it makes a request to a web server much like a user makes a request by typing in a URL into the browser. The data it gets back from the HTTP request is also the same as the data that comes to a browser after searching a URL—only the data is intercepted before in the form of bytes, rather than assembled into a web page visible through a browser.

```
r = requests.get("https://www.nytimes.com/")
```

You can scrape a website with only five lines of code. Below is an example that scraps the home page of *The New York Times* website using the `bs4` and `requests` libraries in python.

```
from bs4 import BeautifulSoup
import requests

r = requests.get("https://www.nytimes.com/")
source = r.content
soup = BeautifulSoup(source, 'html.parser')
```

The `requests` library is one of the most popular libraries in the python programming language. Enabling programs to send and receive data through HTTP networks (i.e., the internet), it is the bedrock for virtually all web scraping and API tasks. The `get()` function from `requests` can access the *NYTimes* homepage, which is saved in the form of bytes via the `content` property. Then, using `bs4`, a python library for working with HTML data, the data is parsed into an object called `soup` (following `bs4`'s naming conventions), which is a pythonic object of HTML data.

The beauty of `bs4` is that it makes HTML—a notoriously fussy language that is the reason why so many web frameworks exist in the first place, attempting to abstract the user from working directly in HTML—accessible through simple python syntax.

For example, the expression `soup.title` returns the following output:

```
>>> soup.title
<title data-rh="true">The New York Times - Breaking News, US News, World News and
```

Similarly, other HTML elements can now be accessed using python dot syntax.

```
>>> soup.p
<p>Make sense of the day's news and ideas.</p>

>>> soup.a
<a class="css-kgn7zc" href="#site-content">Skip to content</a>

>>> soup.h1
<h1 class="css-1dv1kvn">New York Times - Top Stories</h1>
```

Further properties like `text` (for just text) and functions like `find()` are also possible. Functions can be further specified when combined with the `class_` parameter, to search the HTML with more precision. Due to python syntax, they can also be chained together to create complex expressions. These building blocks enable programmers to navigate the HTML tree with granularity.

```
>>> soup.h1.text
'New York Times - Top Stories'

>>> soup.find('div', class_ = 'css-cfnhvx')
<div class="css-cfnhvx" data-tpl="h"><a class="tpl-1bl css-5mgoji" data-tpl="1" href="#">
U.S. and Iran Sink Into Violent Cycle After Trading Strikes</a></div>

>>> soup.find('div', class_ = 'css-cfnhvx').text
'U.S. and Iran Sink Into Violent Cycle After Trading Strikes'
```

HTML classes for an element can be found by using built-in browser tools like the web inspector. On the webpage, right click on the relevant text and select **inspect** from the dropdown menu. The element will already be highlighted, with information for the `class` attribute visible in the highlighted portion.²

²Note that across programming languages like HTML and python, terms like "attributes" mean different things and in practice function very differently. In HTML, an attribute is data contained within an HTML element, like a value specifying a CSS class, such as `<h1 class=title>New York Times</h1>`. By contrast, in python an attribute is information associated with an object model, like `r.content` or `r.status_code`, which is data associated with an HTTP request object, `r`.

